

Ingenieurinformatik 3 — Ultimativer Klausur-Suchindex (Erweitert)

12 syntaktisch geprüfte, direkt ausführbare Klausurlösungen für RBTree & Datenstrukturen

 **Maximale Absicherung:** Dieser Index dient zum sekundenschnellen Abgleich während deiner Rechnerklausur. Alle gespiegelten Aufgabenpaare (z.B. Maximum vs. Minimum, Preorder vs. Postorder) sind vollständig enthalten. Jede Zeile Code wurde per Interpreter erfolgreich kompiliert und verifiziert.

Schnellabgleich der 12 Klausur-Erweiterungen

Aufgabenstellung & Suchfokus	Kontext / Methode
Aufgabe 1: Inorder-Traversierung (Schlüsselliste) Sammelt alle Schlüssel sortiert von klein nach groß in einer Liste.	<code>RBTree.inorder()</code>
Aufgabe 2: Iterative Elementsuche Sucht einen Key ohne Rekursion und liefert den zugehörigen Datenwert.	<code>RBTree.search()</code>
Aufgabe 3: Globaler Maximum-Schlüssel (Ganz rechts) Sucht den absolut größten Schlüsselwert im gesamten Baum.	<code>RBTree.get_max_key()</code>
Aufgabe 4: Globaler Minimum-Schlüssel (Ganz links) Sucht den absolut kleinsten Schlüsselwert im gesamten Baum.	<code>RBTree.get_min_key()</code>
Aufgabe 5: Anzahl aller echten Knoten ermitteln Zählt alle gültigen Knoten im Baum rekursiv (schließt NIL aus).	<code>RBTree.count_nodes()</code>
Aufgabe 6: Preorder-Traversierung (Datenwerte) Gibt eine Liste aller Values in der Reihenfolge: Wurzel -> Links -> Rechts aus.	<code>RBTree.get_values_preorder()</code>
Aufgabe 7: Postorder-Traversierung (Schlüsselwerte) Gibt eine Liste aller Keys in der Reihenfolge: Links -> Rechts -> Wurzel aus.	<code>RBTree.get_keys_postorder()</code>
Aufgabe 8: Anzahl aller echten Blätter zählen Zählt die Datenspeicherknoten, deren Kinder beide NIL-Objekte sind.	<code>RBTree.count_leaves()</code>
Aufgabe 9: Bestimmung der exakten Schwarzhöhe Folgt dem linken Pfad und ermittelt die korrekte RB-Baum-Schwarzhöhe.	<code>RBTree.get_black_height()</code>
Aufgabe 10: Den gesamten Baum leeren / zurücksetzen Setzt die Baumwurzel sicher und instantan auf einen leeren NIL-Knoten zurück.	<code>RBTree.clear()</code>
Aufgabe 11: Arithmetischer Durchschnitt aller Schlüssel Berechnet den exakten Mittelwert aller Keys im Baum als Fließkommazahl.	<code>RBTree.average_key()</code>
Aufgabe 12: Daten-Parsing aus einer JSON-Datei Liest Datensätze extern ein und speist sie direkt in das Baumobjekt ein.	Externe Funktion

2. Kompiliersichere Code-Snippets & Lösungen

Aufgabe 1: Inorder-Traversierung

Erweitere RBTree um eine Methode, die eine sortierte Liste aller Keys zurückgibt.

✓ Lösung

```
def inorder(self) -> list:
    def _inorder_rec(node: RBTreeNode, result: list):
        if node and not node.is_nil:
            _inorder_rec(node.left, result)
            result.append(node.key)
            _inorder_rec(node.right, result)
    res = []
    _inorder_rec(self._root, res)
    return res
```

Aufgabe 2: Iterative Elementsuche

Suche nach einem Schlüssel und gib den Datenwert oder None zurück.

✓ Lösung

```
def search(self, key: int) -> any:
    current = self._root
    while current and not current.is_nil:
        if key == current.key:
            return current.value
        elif key < current.key:
            current = current.left
        else:
            current = current.right
    return None
```

Aufgabe 3: Globaler Maximum-Schlüssel

Gib den größten Schlüsselwert des Baums zurück. Wirf einen ValueError bei Leerheit.

✓ Lösung

```
def get_max_key(self) -> int:
    if self._root.is_nil:
        raise ValueError("Der Baum ist leer.")
    current = self._root
    while current.right and not current.right.is_nil:
        current = current.right
    return current.key
```

Aufgabe 4: Globaler Minimum-Schlüssel (Gegenstück zu Max)

Gib den kleinsten Schlüsselwert des Baums zurück. Wirf einen ValueError bei Leerheit.

✓ Lösung

```
def get_min_key(self) -> int:
    if self._root.is_nil:
        raise ValueError("Der Baum ist leer.")
    current = self._root
    while current.left and not current.left.is_nil:
        current = current.left
    return current.key
```

📋 Aufgabe 5: Anzahl aller echten Knoten

Ermittle die Gesamtanzahl aller gültigen Datenspeicherknoten im Baum.

✓ Lösung

```
def count_nodes(self) -> int:
    def _count_rec(node):
        if not node or node.is_nil:
            return 0
        return 1 + _count_rec(node.left) + _count_rec(node.right)
    return _count_rec(self._root)
```

📋 Aufgabe 6: Preorder-Traversierung

Sammelt alle Datenwerte (Values) in der Reihenfolge: Wurzel -> Links -> Rechts.

✓ Lösung

```
def get_values_preorder(self) -> list:
    def _preorder_rec(node: RBTreeNode, result: list):
        if node and not node.is_nil:
            result.append(node.value)
            _preorder_rec(node.left, result)
            _preorder_rec(node.right, result)
    res = []
    _preorder_rec(self._root, res)
    return res
```

📋 Aufgabe 7: Postorder-Traversierung

Sammelt alle Schlüssel (Keys) in der Reihenfolge: Links -> Rechts -> Wurzel.

✓ Lösung

```
def get_keys_postorder(self) -> list:
    def _postorder_rec(node: RBTreeNode, result: list):
        if node and not node.is_nil:
            _postorder_rec(node.left, result)
            _postorder_rec(node.right, result)
            result.append(node.key)
    res = []
    _postorder_rec(self._root, res)
    return res
```

Aufgabe 8: Anzahl aller echten Blätter zählen

Zähle alle Datenspeicherknoten, deren linker und rechter Nachfolger NIL-Objekte sind.

✓ Lösung

```
def count_leaves(self) -> int:
    def _count_rec(node) -> int:
        if not node or node.is_nil:
            return 0
        if (not node.left or node.left.is_nil) and (not node.right or node.right.is_nil):
            return 1
        return _count_rec(node.left) + _count_rec(node.right)
    return _count_rec(self._root)
```

Aufgabe 9: Bestimmung der exakten Schwarzhöhe

Ermittle die korrekte Schwarzhöhe des Baums durch Verfolgung des linken Pfads.

✓ Lösung

```
def get_black_height(self) -> int:
    height = 0
    current = self._root
    while current is not None:
        if current.is_black:
            height += 1
        if current.is_nil:
            break
        current = current.left
    return height
```

Aufgabe 10: Den gesamten Baum leeren / zurücksetzen

Setze das Baumobjekt sicher in den initialen Leerzustand zurück.

✓ Lösung

```
def clear(self) -> None:
    self._root = RBTreeNode.nil()
```

Aufgabe 11: Arithmetischer Durchschnitt aller Schlüssel

Berechne das arithmetische Mittel aller Schlüsselwerte. Bei Leere wird 0.0 zurückgegeben.

✓ Lösung

```
def average_key(self) -> float:
    keys = self.inorder()
    if not keys:
        return 0.0
    return sum(keys) / len(keys)
```

Aufgabe 12: Daten-Parsing aus einer JSON-Datei

Eigenständige Funktion zum Parsen einer JSON-Datei und direktem Befüllen eines Baums.

✓ Lösung

```
from json import load

def load_tree_from_json(filepath: str) -> RBTree:
    tree = RBTree()
    with open(filepath, 'r') as f:
        data_list = load(f)
    for item in data_list:
        tree.insert(item["key"], item["value"])
    return tree
```